COMP90038 Assignment 2 (15% of the Subject Assessment)

An Experimental Study on Heap

Due Date: **23:59 May 24 (Sunday), 2026**

Prepared by **Junhao Gan**

Background. When writing a research paper, sometimes, in addition to theoretical analysis, it is also important to include a section to demonstrate the *performance in practice* of the proposed algorithms (and data structures) by *experiments*. A good experiment section can often maximize the chance of a paper to be accepted, since it serves as extra convincing evidence for the superiority of the proposition in the paper.

In this assignment, you are asked to conduct an experimental study on Heap and write an experiment section to demonstrate the results as if you are writing a research paper on Heap.

Specifications. There are **three** main tasks in this assignment:

- Task 1: implement a data generator, a binary heap (with a complete binary tree) and a simple competitor (an array);
- Task 2: conduct the required experiments; and
- Task 3: write a report to demonstrate and analyse the experimental results.

Task 1: Implementations.

About Programming Language. There is *no* requirement on any specified programming language in this assignment; you can use whatever programming language you prefer.

The Data Generator. In the experimental study, each *data element* is represented by an *integer* `key`, where `key` indicates the *value* of the element. The data generator should have an interface, called `gen_element()`, to generate a new element by the following steps.

`gen_element()`:

- `key` $\leftarrow$ an integer that is drawn uniformly at random from the range $[0, 10^7]$;
- return `key` as the generated element;

Furthermore, the data generator should also have three more interfaces, `gen_push`(), `gen_pop`() and `gen_getTop`(), to generate a `push`, a `pop` and a `getTop` operation, respectively. Specifically, these operations have the following forms:

- a `push` operation of an element in the form of $(1, \texttt{key})$, where 1 indicates that this operation is a `push`, and `key` is the element to be pushed;
- a `pop` operation in the form of (2), where 2 indicates that this operation is a `pop`;
- a `getTop` operation in the form of (3), where 3 indicates that this operation is a `getTop`;

The detailed steps of these three interfaces are as follows:

`gen_push`():

- $x \leftarrow$ a new element of `key` generated by invoking `gen_element`();
- return $(1, x)$, a `push` for element x;

`gen_pop`():

- return 2, a `pop` operation;

`gen_getTop`():

- return 3, a `getTop` operation;

With these three interfaces, we can generate *operation sequences* with specified lengths. In particular, we can generate a file in the following format:

```
10
1 3
1 7
3
1 9
2
1 1
3
2
1 2
2
```

The number, 10, in the first line indicates the total number of operations in the sequence. And each of the subsequent 10 lines indicates an operation. Specifically, the above file specifies a sequence of 10 operations; they respectively are:

`push`(3): push an element with `key` = 3
`push`(7)
`getTop`()
`push`(9)
`pop`()
`push`(1)
`pop`()
`getTop`()
`push`(2)
`pop`()

The Max-Heap. Implement a max-heap with our complete binary tree version discussed in class. In your implementation, you can assume that ***the total number of elements in the heap is never more than*** 10^6***. Therefore,*** **creating an array of length** 10^6 **suffices for all the experiments in this assignment.**

The max-heap should be able to support the following functionalities:

- `push`(`key`): push an element with `key`
- `pop`();

- `getTop`();
- `heapify`();

According to our lectures, to implement these operations, you will need to implement `bubble-up`(i) and `bubble-down`(i), where i is the index of the node in the array, on which the operation is performed.

The Competitor. The competitor in this experiment is an *array* A of length 10^6. We assume the index of A starts from 0. This array A maintains two information:

- a counter cnt which is initially 0 and is the *current number of elements*in A, and
- an index $i_{\max}$ which is the index of the largest element in A; if A is empty, i.e., $cnt = 0$, then $i_{\max} = -1$.

Moreover, A supports the following operations:

- `push`(`key`):
  - $A[cnt] \leftarrow$ `key`, i.e., append `key` at the back of A;
  - if $i_{\max} = -1$ or $A[i_{\max}] < A[cnt]$, $i_{\max} \leftarrow cnt$;
  - $cnt \leftarrow cnt + 1$;
- `pop`():
  - if $i_{\max} = -1$, return `null`;
  - $\texttt{key}_{\max} \leftarrow A[i_{\max}]$;
  - swap element $A[i_{\max}]$ with $A[cnt - 1]$, the *last* element at the back of the array A;
  - delete $A[cnt - 1]$ by setting $cnt \leftarrow cnt - 1$;
  - if $cnt = 0$, set $i_{\max} \leftarrow -1$;
  - find the *new* maximum element with the largest `key` by scanning the array A from index 0 to $cnt - 1$; let the index of this element be i; update $i_{\max} \leftarrow i$;
  - return $\texttt{key}_{\max}$;
- `getTop`():
  - if $i_{\max} = -1$, return `null`;
  - $\texttt{key}_{\max} \leftarrow A[i_{\max}]$;
  - return $\texttt{key}_{\max}$;

Task 2: Experiments.

Conduct the following *four* experiments.

Exp 1: Time v.s. Length of `push`*-only Sequence.* In this experiment, we study the *total running time* (of the max-heap and the above array A, respectively) v.s. the lengths L of sequences consisting of `push` operations only. Specifically, we **first** generate *five* `push`-only sequences $\sigma_1, \sigma_2, \ldots, \sigma_5$ respectively with length L = 0.1M, 0.2M, 0.5M, 0.8M, 1M, where M stands for *one million*, i.e., 10^6. **Then** we input the **same** $\sigma_1, \ldots, \sigma_5$ to each of the max-heap and the array A.

Exp 2: Time v.s. `getTop` Percentage. In this experiment, we consider a sequence of a *fixed length* $L = 1\text{M}$ of operations (including both `push` and `getTop`). We vary the *in-expectation percentage*, $\%_{\texttt{getTop}}$, of `getTop` in five operation sequences $\sigma_1, \ldots, \sigma_5$, where $\%_{\texttt{getTop}} = 0.1\%, 0.5\%, 1\%, 5\%, 10\%$. Specifically, an operation sequence σ with $\%_{\texttt{getTop}}$ is generated as follows:

- $\sigma \leftarrow \emptyset$;
- for $i = 1$ to $L = 1\text{M}$,
  - generate an operation s_i such that: s_i is a `getTop` with probability $\%_{\texttt{getTop}}$, and it is a `push` with probability $1 - \%_{\texttt{getTop}}$;
  - add s_i to σ;
- return the operation sequence σ;

Run the max-heap and the array A on these sequences $\sigma_1 \ldots, \sigma_5$, and measure their corresponding total running time.

Exp 3: Time v.s. `pop` Percentage. Analogous to Exp 2, we consider a sequence of a *fixed length* $L = 1\text{M}$ of operations which contains `push` and `pop` *only*. We vary the *in-expectation percentage*, $\%_{\texttt{pop}}$, of the `pop` operations in the sequence, where $\%_{\texttt{pop}} = 0.1\%, 0.5\%, 1\%, 5\%, 10\%$. An operation sequence σ with $\%_{\texttt{pop}}$ is generated in an analogous way as the operation sequence with $\%_{\texttt{getTop}}$ in Exp 2. Run both of the max-heap and the array A on these sequences and measure their corresponding overall running time on each of the sequences.

Exp 4: Heapify v.s. Push-One-by-One. In this experiment, we reuse the `push`-only sequences $\sigma_1, \ldots, \sigma_5$ in Exp 1. Compare the overall running time of the following two different construction methods for the max-heap on $\sigma_1, \ldots, \sigma_5$.

Method 1: `heapify`(σ)

- initialise an empty max-heap $\mathcal{H}$;
- scan σ and collect all the elements to be pushed in the array of $\mathcal{H}$;
- heapify $\mathcal{H}$ to be a valid max-heap;
- return $\mathcal{H}$;

Method 2: `push-one-by-one`(σ)

- initialise an empty max-heap $\mathcal{H}$;
- invoke $\mathcal{H}$.`push`(`key`) for each `push` operation in σ one by one;
- return $\mathcal{H}$;

Note: If an algorithm takes more than 3 hours in an experiment, then you may terminate it and record its running time as `Out of Time`.

Task 3: The Report.

You will need to write and submit a *report* on the experimental results. The report should contain the following components:

- *Experiment Environment.* Explain clearly the necessary information for others to *reproduce* your experiments, such as: the CPU frequency, the size of memory, the operating system, the programming language, the compiler (if applicable), etc. **[1 mark out of 15]**

- *Experiments.* **[Each experiment takes 3 marks; thus in total 12 marks out of 15]**

  - *Result Demonstration.* Report the results of the four experiments with diagrams. Specifically, in each diagram, the x-axis represents the variable: L in Exp 1, $\%_{\texttt{getTop}}$ in Exp 2, $\%_{\texttt{pop}}$ in Exp 3, and L in Exp 4. The y-axis is the corresponding overall running time of the max-heap and the array in Exp 1-3 and of the two construction methods in Exp 4. You will need to *plot* the results of the corresponding competitors. Hence, there will be two *lines* in each diagram. **[1 mark out of 3]**
  - *Analysis.* In addition to showing the experimental results, you are asked to *analyse* them. Specifically, you will need to explain, with certain theoretical analysis and complexity, why one algorithm or data structure performs better than the other in some cases while in other cases not. Sometimes, you will also need to analyse the results *across* different diagrams. **[2 marks out of 3]**

- *Conclusion.* Conclude your findings and observations in these experiments. For example, in which cases, which data structure or method is better. **[2 mark out of 15]**

Marking Scheme. The marks for each component in the report are as shown. The grading will be based on the *clarity* and *rigor* of your description or analysis for each part. This is individual work, and you must obey **academic integrity**. However, to ensure scientific integrity, indeed **reproducibility**, your descriptions of your experiments, their design and the design of the data structures must be sufficiently clear that a hypothetical algorithm developer could reproduce the whole experimental study with your report and obtain similar results.

Submissions. You should lodge your submission for Assignment 2 via the LMS (i.e., Canvas). *You must identify yourself in* ***each*** *of your source files and the report.* Poor-quality scans of solutions written or printed on paper will *not* be accepted. There are scanning facilities on campus, not to mention scanning apps for smartphones etc. Solutions generated directly on a computer are of course acceptable. Submit *two* files:

- A `[your-student-id]-report.pdf` file, comprising your report for the experimental study.
- A `[your-student-id]-code.zip` file, containing all your sources files of the implementations for the experiments.

Do not include the testing files, as these might be large. **REPEAT: DO NOT INCLUDE TESTING FILES!** Moreover, it is very important, so that you can justify ownership of your work, that you detail your contributions in comments in your code, and in your report.

Administrative Issues

When is late? What do I do if I am late? The due date and time are printed on the front of this document. The lateness policy is on the handout provided at the first lecture. As a reminder, the late penalty for non-exam assessment is *one* mark per day (or part thereof) overdue. Requests for

extensions or adjustment must follow the University policy (the Melbourne School of Engineering "owns" this subject), including the requirement for appropriate evidence. *We* **DO NOT** *accept extension requests within 3 working days of the due date unless for compelling reasons.*

Late submissions should also be lodged via the LMS, but, as a courtesy, please also email the subject coordinator (Junhao Gan) when you submit late. If you make both on-time and late submissions, please consult the subject coordinators as soon as possible to determine which submission will be assessed.

Individual work. You are reminded that your submission for this Assignment is to be your own individual work. Students are expected to be familiar with and to observe the University's Academic Integrity policy `http://academicintegrity.unimelb.edu.au/`. For the purpose of ensuring academic integrity, every submission attempt by a student may be inspected, regardless of the number of attempts made.

Students who allow other students access to their work run the risk of also being penalized, even if they themselves are sole authors of the submission in question. **By submitting your work electronically, you are declaring that this is your own work.** Automated similarity checking software may be used to compare submissions.

You may re-use code provided by the teaching staff, and you may refer to resources on the Web or in published or similar sources. Indeed, you are encouraged to read beyond the standard teaching materials. However, *all* such sources *must* be cited fully and, apart from code provided by the teaching staff, you must *not* copy code.

Finally. *Despite all these stern words, we are here to help!* There is information about getting help in this subject on the LMS pages. Frequently asked questions about the Assignment will be answered in the LMS discussion group.